

KAVANA RouteFleet

Plataforma de gestión de repartos para empresas de reparto. Permite a los repartidores escanear albaranes, entregar con firma del cliente (POD) y a las oficinas hacer seguimiento de repartos por repartidor, ver firmas y KPIs.

Arquitectura (todo fuera del VPS)

Componente	Donde vive	URL
Backend API (Node/Express, store JSON)	Render	https://routefleet-api.onrender.com
App del repartidor (React PWA)	GitHub Pages	https://kavanasystemsinfo-ui.github.io/kavana-RouteFleet/
Panel oficinas “Torre de Control” (React)	GitHub Pages (rama gh-pages-admin)	https://routefleet.kavanasystems.com (dominio propio)

El VPS solo se usa para desarrollo/documentación. Los proyectos terminados viven en servicios externos (Render + GitHub Pages), igual que CleanStock.

Estructura del repo

server/	Backend Express + store JSON (sin SQLite)
src/db.js	Capa de datos (stops, incidents, drivers, pods)
src/routes/api.js	Endpoints REST
src/services/	pdfService (POD), ocrService, aiService
tests/	29 tests (node --test)
client/	App del repartidor (Vite + React)
client-admin/	Panel de oficinas Torre de Control (Vite + React)
.github/workflows/	deploy.yml (app), deploy-admin.yml (panel)

Desarrollo rápido

```
# Backend
cd server && npm install && npm test          # 29 tests verdes
ROUTEFLEET_DB=/tmp/dev.json PORT=5001 node src/index.js

# App repartidor
cd client && npm install && npm run dev        # http://localhost:5173

# Panel oficinas
cd client-admin && npm install && npm run dev  # http://localhost:5173
```

Deploy

- **Backend:** Render (Web Service, rama main, Root server, start node src/index.js). Variables: VITE_API_BASE no aplica; OFFICE_PIN (PIN oficina, def. 0000), CORS_ORIGINS (def. github.io + routefleet.kavanasystems.com).
- **App repartidor:** GitHub Actions deploy.yml → Pages en /kavana-RouteFleet/. Secret VITE_API_BASE=https://routefleet-api.onrender.com.
- **Panel oficinas:** GitHub Actions deploy-admin.yml → rama gh-pages-admin. En Settings → Pages: rama gh-pages-admin, Custom domain routefleet.kavanasystems.com. DNS: CNAME routefleet → kavanasystemsinfo-ui.github.io.

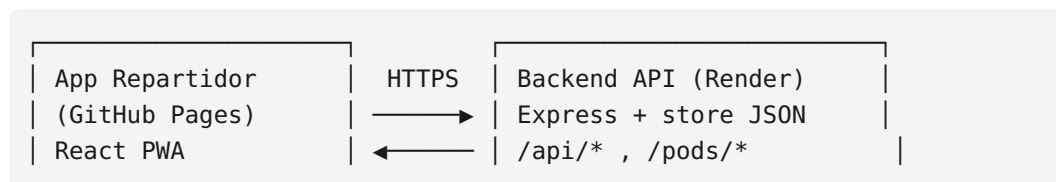
Documentación

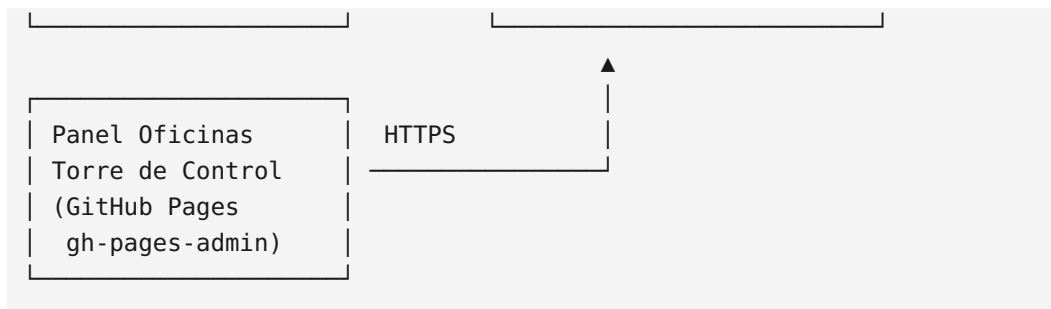
Ver docs/ (ARQUITECTURA.md, BACKEND.md, APP_REPARTIDOR.md, PANEL_OFICINAS.md, DESPLIEGUE.md, API.md). # Arquitectura — KAVANA RouteFleet

Principio rector

El VPS (167.233.97.71) es **solo entorno de desarrollo y documentación**. Los proyectos terminados viven fuera del VPS, igual que CleanStock: - Backend → **Render** (servicio routefleet-api, Free). - Frontends → **GitHub Pages** (sin servidor propio, HTTPS automático).

Componentes





Datos

- Store JSON en disco (`routefleet.json` en el server de Render). Sin SQLite para evitar compilación nativa en Render Free. Listo para migrar a Postgres.
- PODs: PDFs generados con `pdfkit` en `./pods`. En Render son efímeros tras spin-down; el endpoint `GET /api/stops/:id/pod` los regenera al vuelo si la parada tiene firma guardada.

Identidades

- **Repartidor:** se identifica con un PIN (4 dígitos) que se guarda en `localStorage` del móvil. Cada parada se etiqueta con `driver_id`.
- **Oficina:** login con PIN único de empresa (`OFFICE_PIN`, def. 0000). MVP sin usuarios/roles (YAGNI).

Flujo

1. Oficina da de alta repartidores (PIN) en el panel.
2. Repartidor abre la app, introduce su PIN (se guarda en el móvil).
3. Repartidor escanea albarán → parada creada con `driver_id`.
4. Repartidor entrega → POD generado en el navegador (jsPDF) + subido al backend.
5. Oficina ve en el panel: Dashboard (KPIs), Repartos (filtros), Firmas (galería). # Backend — KAVANA RouteFleet

Node/Express, store en JSON (sin SQLite). Todo desplegado en Render (`routefleet-api`). 29 tests con `node --test`.

Arranque

```
cd server
npm install
npm test # 29 tests verdes
ROUTEFLEET_DB=/tmp/dev.json PORT=5001 node src/index.js
```

Variables de entorno

Var	Def.	Uso
PORT	5001	Puerto
ROUTEFLEET_DB	./routefleet.json	Ruta del store JSON
OFFICE_PIN	0000	PIN de login de oficina
CORS_ORIGINS	github.io + routefleet.kavanasystems.com	Orígenes CORS

Capa de datos (src/db.js)

Store JSON con interfaz agnóstica (`initDb()`, `queries.*`):

- stops: [{id, stop_number, address, status, driver_id, receiver_name, signature, created_at}]
- drivers: [{id, name, pin, phone, active}]
- incidents: [{id, stop_id, type, photo_data, notes, created_at}]
- settings: {cost_per_km, cost_per_hour}
- pods: {[stopId]: url}

Servicios

- pdfService.js: genera el POD (PDF) con firma negra sobre blanco.
- ocrService.js: procesa imagen de albarán (stub/IA).
- aiService.js: optimización de ruta (stub/IA). # API REST — KAVANA RouteFleet

Base: <https://routefleet-api.onrender.com/api>

Paradas (stops)

- GET /stops → lista. Query: ? driver_id=1&status=delivered&from=2026-07-01&to=2026-07-31
- POST /ocr_manual → crea parada. Body: {stop_number, address, driver_id}
- PATCH /stops/:id → actualiza. Body: {status, signature, receiverName, driver_id}. Si status=delivered y signature, genera POD y devuelve {success, pod_url}.
- DELETE /stops/:id · DELETE /stops (borra todas)
- POST /stops/:id/incident → Body {type, photo_data, notes} (pone status=incident)
- GET /stops/:id/pod → {pod_url} (regenera si hace falta)

Repartidores (drivers)

- GET /drivers → lista
- POST /drivers → Body {name, pin, phone} (404 si falta name/pin)
- PATCH /drivers/:id → Body {active: bool}

Oficina

- POST /office/login → Body {pin}. 200 si coincide con OFFICE_PIN, 401 si no.

Config / métricas

- GET /settings · PUT /settings ({cost_per_km, cost_per_hour})
- GET /dashboard-data → {metrics, stops (con pod_url), settings}

PODs (estáticos)

- GET /pods/<file>.pdf → descarga del PDF # App del Repartidor — KAVANA RouteFleet

React PWA (carpeta client/). Desplegada en GitHub Pages en /kavana-RouteFleet/. Estética oscura industrial, pensada para móvil.

Pantalla de identificación (PIN)

Al abrir la app, si no hay PIN guardado, pide el PIN del repartidor. Se valida contra GET /api/drivers y se guarda en localStorage (routefleet_driver_id, routefleet_driver_name). No se pide más hasta cambiar de dispositivo o pulsar “cambiar”.

Funciones

- **Carga:** escáner de albarán (cámara) → POST /api/ocr (IA) → crea parada con driver_id vía POST /api/ocr_manual.
- **Mapa:** vista de la parada activa (Google Maps embed).
- **Entregar:** firma del cliente en canvas (blanco, trazo negro) → PATCH /stops/:id con firma → genera POD en el navegador (jsPDF) y descarga garantizada (DESCARGAR POD). También sube la firma al backend.
- **Incidencia:** foto + nota → POST /stops/:id/incident.

Variables

- VITE_API_BASE (build-time) → `https://routefleet-api.onrender.com` Sin slash final; el cliente añade `/api`.

Build

```
cd client && npm install && npm run build # → dist/
```

Panel de Oficinas — Torre de Control

React (carpeta `client-admin/`). Desplegado en GitHub Pages en la rama `gh-pages-admin`, con dominio propio `routefleet.kavanasystems.com`.
Estética backoffice clara, sidebar + tablas densas (pensado para escritorio).

Login

Pantalla de PIN de oficina (POST `/api/office/login`). PIN por defecto `0000` (cambiable en Render con `OFFICE_PIN`).

Secciones

- **Dashboard:** KPIs (total / entregados / pendientes / incidencias / OPEX estimado) + barras de entregas por repartidor (%).
- **Repartidores:** alta (nombre, PIN, teléfono) y activar/desactivar.
- **Repartos:** tabla con filtros por repartidor, estado y rango de fechas. Enlace a POD por parada.
- **Firmas:** galería de firmas de clientes (iframe del POD + descarga PDF), filtrable por repartidor y fechas.
- **Incidencias:** lista de incidencias reportadas por los repartidores.

Variables

- VITE_API_BASE (build-time) → `https://routefleet-api.onrender.com`

Build

```
cd client-admin && npm install && npm run build # → dist/
```

Despliegue — KAVANA RouteFleet

1. Backend (Render)

- New → Web Service → repo kavana-RouteFleet, rama main, Root server.
- Build: `npm install` · Start: `node src/index.js`.
- Variables: `OFFICE_PIN` (opcional), `CORS_ORIGINS` (opcional, ya trae defaults).
- **Importante:** el auto-deploy puede quedar fijado a un commit viejo. Tras push de cambios al backend, pulsa **Manual Deploy** para aplicar main HEAD.

2. App repartidor (GitHub Pages)

- Workflow `.github/workflows/deploy.yml` (Source: GitHub Actions).
- Secret repo `VITE_API_BASE=https://routefleet-api.onrender.com`.
- URL: `https://kavanasystemsinfo-ui.github.io/kavana-RouteFleet/`
- Base del build: `/kavana-RouteFleet/` (en `client/vite.config.js`).

3. Panel oficinas (GitHub Pages + dominio propio)

- Workflow `.github/workflows/deploy-admin.yml` publica `client-admin/dist` en la rama `gh-pages-admin` (orphan).
- En repo → Settings → Pages: selecciona rama `gh-pages-admin`, carpeta `/` (root).
- Custom domain: `routefleet.kavanasystems.com` (activa HTTPS).
- DNS (gestor de `kavanasystems.com`): CNAME `routefleet` → `kavanasystemsinfo-ui.github.io`.

Verificar tras desplegar

Backend vivo

```
curl https://routefleet-api.onrender.com/api/settings
```

CORS para panel

```
curl -s -i https://routefleet-api.onrender.com/api/drivers \
  -H "Origin: https://routefleet.kavanasystems.com" \
  | grep access-control-allow-origin
```